INTERVIEW-READY FIELD GUIDE

The 4 Principles of Object-Oriented Programming

Definitions and real examples from your MechanicShop project

PROTECT	**Encapsulation**
PROMISE	**Abstraction**
PARENT	**Inheritance**
PLUG-IN	**Polymorphism**

A well-designed object protects its state, exposes a promise, can inherit from a parent, and lets implementations plug into the same contract.

Built for recall, technical discussion, and confident interview answers.
MechanicShop Workshop / C# / Domain-driven application architecture

01 Encapsulation / Protect

Protect an object's valid state and control how that state changes.

DEFINITION

Encapsulation bundles data and behavior inside an object, restricts direct access to its internal state, and provides controlled operations that preserve business rules.

> **MEMORY HOOK** The object is the guardian of its own rules. Do not hand strangers the keys to its state.

MECHANICSHOP EXAMPLE

```
public WorkOrderState State { get; private set; }

public Result<Updated> UpdateState(WorkOrderState newState)
{
    if (!CanTransitionTo(newState))
        return WorkOrderErrors.InvalidStateTransition(State, newState);

    State = newState;
    return Result.Updated;
}
```

MechanicShop.Domain/Workorders/WorkOrder.cs

PROTECTED STATE

Callers can read State but cannot assign an arbitrary value.

CONTROLLED CHANGE

UpdateState validates the transition before modifying the entity.

MORE EVIDENCE

Private constructors, factory methods, and read-only collection views.

INTERVIEW ANSWER

> ***Encapsulation means protecting an object's internal state and requiring changes to pass through controlled methods. In my MechanicShop project, WorkOrder.State has a private setter and can only change through UpdateState, which validates the transition before modifying the entity.***

PROFESSIONAL PHRASE The entity enforces its own invariants instead of relying on every caller to behave correctly.

02 Abstraction / Promise

Expose essential capabilities while hiding implementation details.

DEFINITION

Abstraction presents a simple contract that describes **what** something can do, while keeping the caller independent from **how** that capability is implemented.

> **MEMORY HOOK** Press the button; you should not need to understand the wiring behind the wall.

MECHANICSHOP EXAMPLE

```
public interface IWorkOrderNotifier
{
    Task NotifyWorkOrdersChangedAsync(
        CancellationToken ct = default);
}

public sealed class SignalRWorkOrderNotifier : IWorkOrderNotifier
{
    // SignalR details live here, behind the contract.
}
```

Application/Common/Interfaces/IWorkOrderNotifier.cs + Infrastructure/RealTime/SignalRWorkOrderNotifier.cs

CALLER SEES	CALLER IGNORES	DESIGN RESULT
A capability named NotifyWorkOrdersChangedAsync.	SignalR hubs, clients, transport details, and configuration.	The application layer stays independent from infrastructure.

DO NOT CONFUSE THESE

ENCAPSULATION	ABSTRACTION
Protects the inside of an object.	**Hides unnecessary details from callers.**

INTERVIEW ANSWER

> ***Abstraction exposes essential capabilities while hiding implementation details. My application depends on IWorkOrderNotifier, so it knows that work-order changes can be announced without knowing that SignalR performs the notification.***

03 Inheritance / Parent

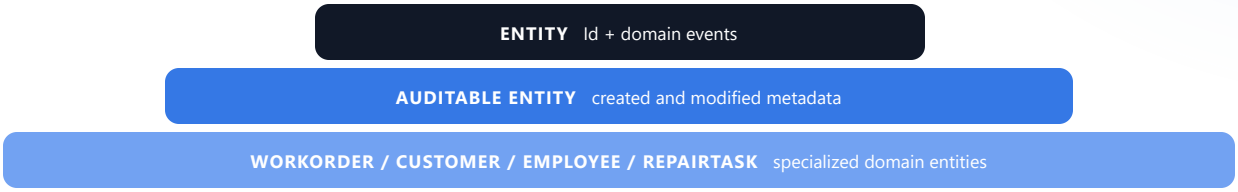
Create a specialized type from a general type when a true is-a relationship exists.

DEFINITION

Inheritance allows a derived class to receive shared state or behavior from a base class. It is most appropriate when the derived type genuinely is a specialized form of the base type.

> **MEMORY HOOK** A child receives family traits, but it still has its own identity and responsibilities.

MECHANICSHOP HIERARCHY




```
public abstract class Entity
{
    public Guid Id { get; }
}

public abstract class AuditableEntity : Entity
{
    public DateTimeOffset CreatedAtUtc { get; set; }
}

public sealed class WorkOrder : AuditableEntity
{
    // Work-order-specific state and behavior
}
```

Domain/Common/Entity.cs + AuditableEntity.cs + Workorders/WorkOrder.cs

WHY IT FITS

A WorkOrder really is an auditable entity.

SHARED TRAITS

Identity and audit metadata live in the base hierarchy.

JUDGMENT

Prefer composition when the relationship is has-a, not is-a.

INTERVIEW ANSWER

> ***Inheritance models an is-a relationship and centralizes genuinely shared functionality. In my project, WorkOrder is an AuditableEntity, so it inherits identity and audit information while adding work-order-specific behavior.***

04 Polymorphism / Plug-in

Use different implementations through the same contract without changing the caller.

DEFINITION

Polymorphism lets code communicate through a common base type or interface. Concrete behavior can vary at runtime while the caller continues to use the same operation.

MEMORY HOOK One socket, many compatible devices. The caller uses the socket, not the internal electronics.

MECHANICSHOP EXAMPLE

```
// The handler depends on the contract:
CreateWorkOrderCommandHandler(IWorkOrderPolicy workOrderPolicy)

// Production selects this implementation:
services.AddScoped<IWorkOrderPolicy, WorkOrderPolicy>();

// A test can provide another implementation:
IWorkOrderPolicy policy = new FakeWorkOrderPolicy();
```

CreateWorkOrderCommandHandler.cs + Infrastructure/DependencyInjection.cs

SAME CALLER

The handler always communicates through IWorkOrderPolicy.

DIFFERENT BEHAVIOR

Production and tests can supply different implementations.

RUNTIME CHOICE

The dependency-injection container selects the implementation.

ABSTRACTION VS POLYMORPHISM

The interface is the abstraction. The ability to substitute WorkOrderPolicy, FakeWorkOrderPolicy, or a future implementation through that interface is polymorphism.

INTERVIEW ANSWER

> ***Polymorphism allows code to work with a contract instead of a specific implementation. My command handlers depend on IWorkOrderPolicy, while dependency injection supplies WorkOrderPolicy at runtime. Tests can substitute a fake without changing the handler.***

PROFESSIONAL PHRASE The caller is decoupled from the concrete implementation.

FINAL INTERVIEW SHEET

Explain All Four With Confidence

Connect each principle to a concrete design decision, then explain why that decision improves the system.

30-SECOND INTERVIEW ANSWER

> ***I apply all four OOP principles in my MechanicShop project. WorkOrder uses encapsulation through private setters and validated methods. Interfaces such as IWorkOrderPolicy provide abstraction. WorkOrder inherits shared identity and audit data from AuditableEntity. Finally, dependency injection gives us polymorphism by allowing different IWorkOrderPolicy implementations without changing the handlers.***

THE FOUR-QUESTION TEST

ENCAPSULATION	**Who is allowed to change this object's state?**	The object, through controlled behavior.
ABSTRACTION	**What does the caller need to know?**	The contract, not implementation details.
INHERITANCE	**Is this truly a kind of the base type?**	Use it for a meaningful is-a relationship.
POLYMORPHISM	**Can another implementation replace this one?**	Yes, through the same contract.

SAY IT LIKE A PROFESSIONAL

INSTEAD OF	"I made the properties private."
SAY	"The entity protects its invariants through controlled state transitions."
INSTEAD OF	"I used an interface."
SAY	"The application depends on an abstraction, enabling interchangeable implementations."

NEVER FORGET

ENCAPSULATION PROTECTS.
ABSTRACTION SIMPLIFIES.
INHERITANCE SPECIALIZES.
POLYMORPHISM SUBSTITUTES.